1)

- Тип client_id можно поменять на int (этот тип обеспечивает быструю индексации, легко настраивается для автоинкремента, также обеспечивает эффективное использование памяти по сравнению с number). Если он является первичным ключом, то нужно убедиться, что он автоматически инкрементируется и является уникальным.
- client_name и client_surname: можно ограничить длину до более разумного значения, например varchar(50), это поможет избежать избыточности использования места
- login: нет указания на уникальность. Это может привести к конфликтам при аутентификации. Поэтому нужно добавить ограничение UNIQUE на поле login
- city_id: нужно убедиться, что существует таблица городов, на которую ссылается этот идентификатор
- age: Так как возраст меняется каждый год, то лучше хранить дату рождения клиента (типа date), что поможет вычислять возраст клиента на текущий момент
- reg_date: нужно проверить, что поле автоматически заполняется при регистрации клиента

2)

- В таблице items поле item_cost следует удалить, так как цена товара со временем меняется, а хранение только текущей стоимости не позволяет вести историю изменений.
- Тип item_id лучше поменять на int для облегчения настройки автоинкремента и эффективной индексации.
- Нужно создать дополнительную таблицу, которая будет хранить историю цен, например, item_price_history. Она будет состоять из:
    - item_id (int) – внешний ключ (идентификатор товара), который ссылается на товар в таблице items
    - price (decimal(10,2)) - стоимость товара на определенную дату. Хранит историческую стоимость товара
    - valid_from (date) - дата, с которой начинает действовать данная цена
    - valid_to (date) - дата, окончания действия цены (если null – цена считается актуальной)
- Логика работы:
    - Таблица items хранит только основную информацию о товаре, без цены
    - Все изменения цен записываются в отдельную таблицу item_price_history
    - При установке новой цены предыдущая запись закрывается (указывается дата окончания её действия), а новая добавляется с датой начала действия и пустым valid_to (значит цена актуальна)
    - Текущую цену можно найти по признаку valid_to IS NULL
    - Для построения отчетов за определенный период используется таблица item_price_history. Можно определить, какая стоимость была актуальна в любой момент времени, используя интервал действия цены - дату начала (valid_from) и дату окончания (valid_to). Если нужная дата находится между valid_from и valid_to (включительно), либо больше или равна valid_from, когда valid_to равен NULL, значит, именно эта цена была актуальна в тот момент

3)

```
SELECT
        D.DEPARTMENT_ID,
        D.DEPARTMENT_NAME,
        SUM(E.SALARY) AS TOTAL_SALARY
FROM
        EMPLOYEES E
        JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
        JOIN LOCATIONS L ON D.LOCATION_ID = L.LOCATION_ID
WHERE
        E.MANAGER_ID IS NULL
        AND L.CITY = 'SEOUL'
GROUP BY
        D.DEPARTMENT_ID,
        D.DEPARTMENT_NAME
HAVING
        SUM(E.SALARY) > 100000;
```

- Для соединения таблиц используем JOIN, это делает запрос более читаемым, а также он необходим для соединения с таблицей LOCATIONS
- Агрегатная функция SUM не может использоваться в WHERE, поэтому используем её в HAVING, ведь она применяется к результатам группировки
- Используем IS NULL для проверки отсутствия менеджера
- Группировка данных должна быть по DEPARTMENT_ID, а не только по DEPARTMENT_NAME, иначе данные могут агрегироваться некорректно

4) Если карандаш положили вплотную к стене, то как бы человек не старался, перепрыгнуть у него не получится